## From engine data to maintenance decisions

# AeroTwin OS

## Autonomous Predictive Maintenance

An integrated research prototype for health, remaining life, failure risk and maintenance prioritization.

**OVERALL PRODUCT / VERIFIED EVIDENCE / COMMERCIALIZATION**

# The problem: signals do not equal decisions

## Fragmented evidence

Sensor trends, life estimates and risk scores can point in different directions.

## Competing priorities

Early interventions consume resources; late interventions can disrupt operations.

## Limited traceability

A recommendation needs a reason, a confidence signal and an engineer review path.

**Product goal: turn separate predictions into a prioritized, explainable work queue.**

# One product, four connected components

## 01 RUL prediction

How much operating life remains?

## 02 Failure risk

How urgent is the risk over 10 / 30 / 50 cycles?

## 03 Health + anomalies

What changed, where, and why?

## 04 Maintenance supervisor

What should the team do next?

**Unified output: a recommended action with priority, confidence and supporting evidence.**

Backend/main.py; Integration/routes.py; supervisor_bridge.py

# Designed around the maintenance engineer

**1**

## Upload

C-MAPSS .txt or prepared .csv

**2**

## Analyze

Validate and run the four components

**3**

## Review

Compare trends, risk and explanations

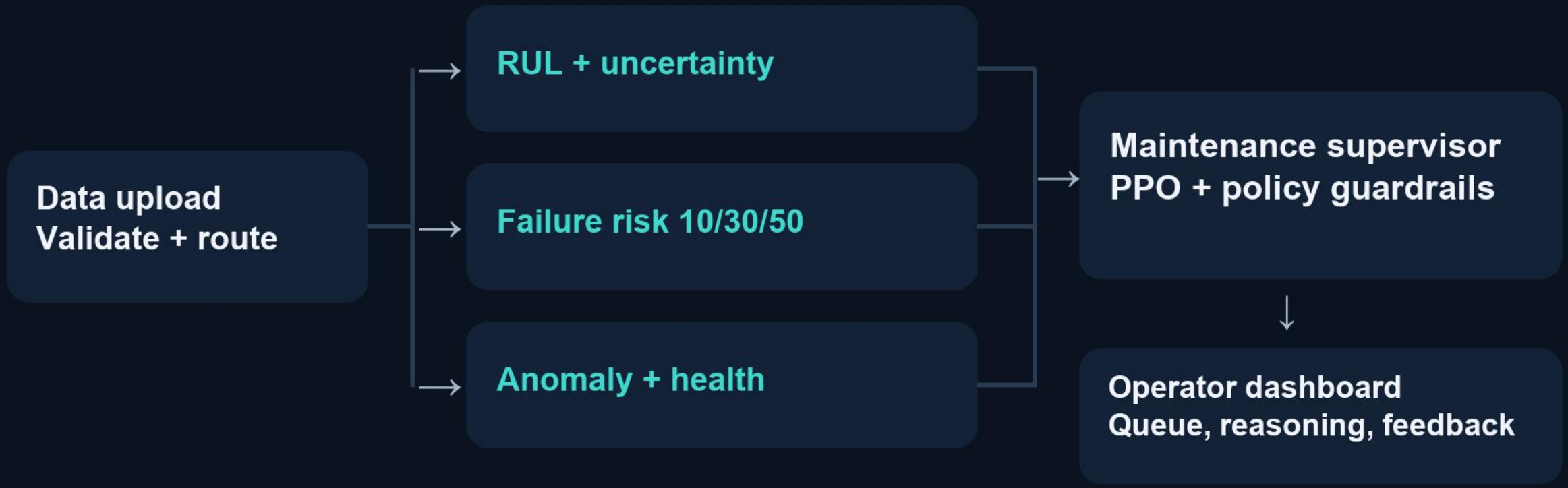
**4**

## Prioritize

Inspect the supervisor work queue

**Feedback and review connect the recommended action to the engineer’s judgement.**

# System architecture: evidence to action




React + TypeScript UI | FastAPI services | Model artifacts + CSV/JSON exchange

Backend/app/services/Integration; Backend/main.py; Frontend/package.json

# Evidence base: NASA C-MAPSS

**4**

C-MAPSS subsets

**21**

Sensor channels

**3**

Operating settings

**Simulated engine trajectories; operating cycles are not hours.**

Validation is retrospective. Real fleet data, maintenance outcomes and prospective testing are still needed.

Local C-MAPSS reader + forecast_target_080.json; NASA C-MAPSS documentation

# RUL prediction: how much life remains?

## Implemented output

Temporal life estimate, degradation trajectory, lifecycle state and prediction interval.

## Where it helps

Supports inspection planning and highlights accelerating or unstable degradation. Engineers see the estimate alongside risk and health.

**Saved pooled evaluation: $R^2$ 0.3954 | RMSE 37.69 cycles | 627 engines**

Predictive_Maintenance/rul_lifecycle_service.py; saved evaluation_metrics.json

# Failure risk: how urgent is the concern?

| Horizon | Decision context | Saved FD002 F1 | Saved recall |
|---|---|---|---|
| 10 cycles | Immediate attention | 0.4800 | 0.5455 |
| 30 cycles | Near-term planning | 0.7419 | 0.8846 |
| 50 cycles | Earlier preparation | 0.8250 | 0.8462 |

**BiLSTM prediction + uncertainty + calibrated thresholds; these figures are not all-subset metrics.**

Failure_Risk FD002 metadata FINAL; failure_risk_service.py

# Health monitoring: explain the deviation

## Frozen monitoring reference

Context-aware healthy expectations drive residuals, condition detection, health index and subsystem reasoning.

## Adaptive sensor forecast

Prior readings improve predictions of actual sensor behavior. Forecast errors remain separate from monitoring residuals.

**Active raw-sensor forecast $R^2$: 0.6417 → 0.6603. Frozen monitoring behavior is preserved.**

Anomaly_Health_Monitoring/config.py; ensemble_twin.py; verified forecast evaluation

# Supervisor: recommend the next action

| | | | | |
|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 5 |
| **Continue** | **Monitor** | **Inspect** | **Maintain** | **Urgent** |
| Operation remains routine | Increase attention | Plan targeted inspection | Schedule maintenance | Immediate maintenance review |

**Inputs are fused with confidence, conflict checks and policy overrides before operator review.**

# Operator workspace: one view of the fleet

## Fleet overview

Health state, RUL distribution, queue and drift

## Engine investigation

Sensor traces, risk horizons and health history

## Maintenance review

Action, priority, confidence and explanation

## Evidence + feedback

Reports, model metrics and engineer response

**Schematic of implemented UI capabilities; each view reads backend outputs.**

Frontend/src/app/routes.tsx; Overview.tsx; MaintenanceSupervisor.tsx

# Measured results: keep the metrics distinct

| Evaluation | Before | Active / saved result |
|---|---|---|
| Raw-sensor forecast $R^2$ | 0.6417 | 0.6603 |
| Raw-sensor RMSE | 0.4318 | 0.4205 |
| Raw-sensor MAE | 0.2482 | 0.2383 |
| RUL prediction (pooled) | Separate task | $R^2$ 0.3954; RMSE 37.69 cycles |

**The requested official raw-sensor $R^2$ of 0.8 has not been reached.**

forecast_target_080.json; Predictive_Maintenance/evaluation_metrics.json

# Supervisor economics: simulation evidence

**28**

Offline test engines

**0 / 28**

Simulated failures

**78.98%**

Simulated cost reduction

**Comparison: reactive baseline. Rule-based policy also had zero simulated failures.**

**Simulation outcomes depend on assumed costs and downtime; customer ROI is not validated.**

# What is validated, and what still needs work

## Verified in the local project

150 forecast/regression tests pass.
Normal retraining reproduces the candidate.
Monitoring outputs remain unchanged.

## Before operational adoption

Improve multi-condition RUL accuracy and interval calibration. Validate real fleet data, alert burden and workflow integration.

**Current maturity: integrated research prototype, ready for a controlled advisory pilot.**

# From prototype to a deployable pilot

## Data + model readiness

Prospective holdout data, sensor mapping, missing-data tests, calibration and drift checks.

## Operational readiness

Authentication, audit logging, monitoring, deployment packaging and measured latency.

## Workflow readiness

Engineer approval, maintenance records, work-order integration and accountable ownership.

**Pilot release gate: evidence quality + workflow safety + reproducible deployment.**

# Evidence used in this presentation

## Implementation

FastAPI integration + four component services
React / TypeScript operator workspace

## Measured evidence

Locked forecast evaluation; saved RUL metrics; FD002 risk metadata; offline supervisor policy evaluation.

**External context: NASA C-MAPSS / IBM Maximo APM / Airbus Skywise**

# Commercialization: start with one buyer

## Initial buyer

MRO or fleet reliability manager with usable engine data, a defined triage problem and an engineer champion.

## Initial offer

A bounded advisory pilot: one fleet or asset group, one workflow, one baseline and clearly measured outcomes.

**Positioning hypothesis: explainable maintenance triage with a practical integration path.**

Proposed positioning; official IBM Maximo APM and Airbus Skywise descriptions

# Commercial offer and proposed pricing

## Paid pilot

8–12 weeks
$5k–$15k
Data audit, shadow evaluation and evidence report.

## Subscription hypothesis

$1,000/site/month
+ $20/engine/month
50 engines: $24k annual recurring revenue.

## Enterprise options

Partner-specific quote for private hosting, integrations, support and assurance work.

**Pricing is illustrative USD; no customers, revenue or willingness to pay are claimed.**

# Go to market through evidence, then expand

## Months 0–3

Interview users.
Secure one design partner.
Agree scope, baseline, data access and review criteria.

## Months 3–6

Run an advisory shadow pilot.
Measure lead time, alert burden and planner effort.
Review every intervention.

## Months 6–12

Convert if the pilot passes.
Package repeatable onboarding.
Test with two further partners.

**Pilot exit: prospective evidence, engineer acceptance, buyer approval and a viable delivery cost.**

Proposed 12-month launch plan; milestones are targets

# Customer ROI: an explicit testable model

| Illustrative first-year input | Calculation | Value |
|---|---|---|
| Avoided disruption value | 2 × $25,000 | $50,000 |
| Planner time value | 200 hours × $50 | $10,000 |
| Subscription + setup | $24,000 + $10,000 | $34,000 |
| Net benefit / modeled ROI | $60,000 − $34,000 | $26,000 / 76.5% |

**Without verified avoided events, this example loses $24k in year one. Validate attribution.**

Illustrative assumptions only; replace every input with customer-validated values

# Commercialization plan: earn the right to scale

## 1 Secure the partner

One scoped buyer problem, usable historical data, engineer sponsor and a paid pilot agreement.

## 2 Prove operational value

Prospective evaluation, reviewed actions, acceptable alert burden, security checks and auditable ROI.

## 3 Scale a repeatable offer

Convert the pilot, standardize onboarding and support, then expand to additional fleets.

**Next milestone: one paid design-partner pilot with success and stop criteria agreed in advance.**